IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE & MANAGEMENT

Project – HighLevel Design
on
AI-DRIVEN SELF-HEALING CLOUD FOR HEALTHCARE

Student Name: FATHIMATH NAZIMA PK

Industry Mentor: SUMIT KUMAR **Guided by:** Ms.MANJULA

Table of Contents

1 Introduction
1.1 Purpose
1.2 scope
1.3 System overview

2 ARCHITECTURE OVERVIEW
2.1 High Level Architecture
2.2 System Modules
2.3 Data Flow
2.4 Deployment View

3 Functional Overview
3.1 Monitoring
3.2 Self-Healing
3.3 Alerting
3.4 Dashboard

4 Technology Stack

5 Non Functional Requirements

6 Future Enhancements

7 reference

INTRODUCTION

1.1 Purpose

This document provides the high-level design of an AI-driven self-healing cloud monitoring system for healthcare service simulation.

The system is designed to:

Monitor service availability
Detect failures automatically
Trigger self-healing actions
Alert users of critical incidents
Visualize system health through a dashboard

1.2 Scope

The scope covers:

Overall architecture
Major system components
Component interactions
Data flow and deployment model

This document focuses on system-level design rather than implementation detail.

1.3 System Overview

The system simulates cloud infrastructure monitoring for healthcare services.

Demo services:

Patient Record Service (simulated by Notepad)
Billing/Support Service (simulated by Calculator)

The system provides:

Process monitoring
Automatic recovery
Risk-based alerting
Health analytics dashboard

ARCHITECTURE OVERVIEW

2.1 High Level Architecture

```
+----------------------+
| Configured Services |
+----------------------+
            |
            v
+----------------------+
| Monitoring Engine |
+----------------------+
            |
Failure Detected
            |
            v
+----------------------+
| Self-Healing Engine |
+----------------------+
      |        |
      |        |
      v        v
Alerts       Logging
        \      /
         v   v
+----------------------+
| Analytics Dashboard
| +----------------------+
```

2.2 System Modules

Monitoring Module

Responsibilities:

Service health checks

Process monitoring

Failure detection

Input:

Configured service list

Output:

Service status
Self-Healing Module

Responsibilities:

Restart failed services
Recovery actions

Input:

Failure event

Output:

Restored service
Risk Analytics Module

Responsibilities:

Failure counting
Risk classification
Stability scoring

Outputs:

LOW
MEDIUM
HIGH
UNSTABLE
Alert Module

Provides:

Popup alerts
Email notifications

Triggers:

Failures
High-risk conditions
Dashboard Module

Provides visualization for:

Service health
Failure metrics
Incident feed
Stability score
Persistence Layer

Stores:

Logs
Reports
Service statistics

2.3 Data Flow

Services
↓
Monitoring
↓
Failure Detection
↓
Recovery
↓
Logs + Stats
↓
Dashboard

2.4 Deployment View

Prototype deployment:

Local Machine
├── **Python Runtime**
├── **Self-Healing Engine**
├── **Streamlit Dashboard**
├── **Logs/Stats Storage**
└── **Scheduler/Loop**
Future deployment:

Cloud VM / Container

3 FUNCTIONAL OVERVIEW

3.1 Monitoring

Periodic checks:

Check process
Healthy ?
Yes → log healthy
No → trigger recovery

3.2 Self-Healing

Recoveryworkflow:

Detect Failure
↓
Classify Risk
↓
Restart Process
↓
Confirm Recovery

3.3 Alerting

Notification channels:

Local popup alerts
Email alerts
Incident reporting

3.4 Dashboard

Provides:

Real-time metrics
Service matrix
Failure charts
Critical incidents
Stability monitoring

4 TECHNOLOGY STACK

Layer	Technology
Language	Python
Monitoring	psutil
Recovery	subprocess
Alerts	tkinter, SMTP
Dashboard	Streamlit
Storage	JSON, Log files
Scheduling	Cron / Interval Loop

5 NON FUNCTIONAL REQUIREMENTS

Availability
System designed for continuous monitoring.

Goal:

High uptime through automated recovery
Reliability

Repeated failures trigger:

risk escalation
unstable classification
Scalability
Supports adding more services through config.
Maintainability

Modular architecture:

independent components
easy enhancements
Performance

Lightweight:

low CPU overhead
fast checks
Security

Current:

Local execution
App password email authentication

Future:

Authentication
Encryption

6 FUTURE ENHANCEMENTS

Possible improvements:

Real healthcare service integration
Predictive failure detection (ML)
Cloud deployment
REST monitoring APIs
Advanced anomaly detection
Container orchestration integration

Example future:

Kubernetes self-healing pods

7 REFERENCES

Python Documentation
Streamlit Documentation
psutil Documentation
SMTP Documentation

High Level Design Diagram (Use in PDF)
Use this cleaner one:

```
+----------------+
| Config Services |
+-----------------+
|
v
+----------------+
| Monitoring Core |
+----------------+
```

```
|
Failure Detection|
v
+----------------+
| Self Healing |
+----------------+
| | |
| | |
v v v
Alerts Logs Analytics
|
v
+--------------+
| Dashboard |
+--------------+
```